



HUST



26/06/2026

Projet Smart système 4A ISIE

Federated Learning pour la Détection de Véhicules et Piétons
sur Systèmes Embarqués

Université de Tours

École Polytechnique de l'université de Tours

Informatique et Systèmes Intelligents Embarqués

Etudiants : Hugo BRAY / Aurelien PAPILLON

Tuteur : Mr. Pierre GAUCHER & Ph.D Thanh-Hai Tran & Ph.D Hai vu

Table des matières

Remerciement.....	4
Introduction	5
1 Présentation générale et cadrage du projet.....	6
1.1 Analyse des besoins et objectifs stratégiques.....	6
1.2 Cahier des charges et exigences du sujet.....	6
2 Conception de l'architecture distribuée.....	10
2.1 Topologie du réseau d'Apprentissage Fédéré (FL).....	10
2.2 Apprentissage centralisé : Établissement du modèle de référence.....	11
2.3 Phase de prototypage : de la simulation au client unique	12
3 Gestion des données et cycle d'apprentissage	14
3.1 Évaluation initiale et limites de YOLOv8n.....	14
3.2 Migration technologique vers YOLOv5.....	14
4 Gestion des données et cycle d'apprentissage	15
4.1 Évaluation et Analyse des Résultats.....	15
4.2 Implémentation du protocole Flower et de l'algorithme FedAvg.....	16
5 Résolution des problèmes et optimisations matérielles.....	19
5.1 Scénario de mise en route du réseau physique	19
5.2 Analyse des anomalies et crashes matériels.....	19
6 Déploiement, Évaluation et Analyse des Résultats	22
6.1 Solutions techniques et configurations appliquées sur les cartes embarqués.....	22
6.2 Évaluation des performances après optimisation	22
7 Bilan de gestion de projet.....	24
7.1 Suivi, planification temporelle et jalons (Gantt)	24
7.2 Organisation et communication.....	24
8 Conclusion.....	25

Remerciement

Nous tenons à exprimer notre gratitude envers toutes les personnes qui ont contribué à la réalisation de ce projet.

Nous remercions tout d'abord nos encadrants, M. Pierre Gaucher, Ph. Thanh-Hai Tran et Ph. Hai Vu, pour leur disponibilité, leurs conseils avisés et leur suivi. Leur expertise a été déterminant dans l'orientation de nos choix techniques et dans la réussite de ce projet.

Nous remercions également l'Université de Tours et Polytech Tours pour nous avoir offert le cadre et les ressources nécessaires à la conduite de ce travail, ainsi que l'Université HUST Vietnam pour cette collaboration internationale enrichissante qui a rendu possible le déploiement de notre système sur ses installations.

Introduction

Dans le cadre de notre formation d'ingénieurs à Polytech Tours, au sein de la filière Informatique et Systèmes Intelligents Embarqués, nous menons un projet axé sur les technologies de l'intelligence artificielle distribuée. Ce projet, réalisé en collaboration internationale avec l'université vietnamienne HUST, s'intitule « *Federated Learning* pour la Détection de Véhicules et Piétons sur Systèmes Embarqués ».

L'objectif principal de ce travail est de concevoir, d'implémenter et d'évaluer un système complet d'Apprentissage Fédéré (*Federated Learning* - FL) de bout en bout. Dédié à la détection en temps réel de véhicules et de piétons, ce système vise à s'affranchir des contraintes de l'approche Cloud centralisée classique, notamment en matière de saturation de la bande passante et de respect de la confidentialité des données privées. Pour répondre à cette problématique, nos recherches exploitent les mécanismes du calcul embarqué (*Edge Computing*) : les données brutes restent cloisonnées sur les terminaux et seul un modèle de vision léger s'entraîne de manière collaborative en échangeant ses paramètres avec un serveur d'orchestration central.

Sur le plan technique, la démarche s'articule autour d'algorithmes de détection de la famille YOLO et d'une infrastructure réseau distribuée, avec pour finalité un déploiement et une évaluation sur des cibles matérielles contraintes (Edge devices).

Ce rapport présente la démarche retenue et la topologie de notre architecture. Il détaille la progression du projet, depuis les phases initiales de simulation sur ordinateur jusqu'à l'analyse des performances réelles sur cartes embarquées (scores de précision, latence d'inférence, overhead de communication et ressources matérielles). Enfin, il dresse un bilan comparatif entre l'approche centralisée et fédérée, avant de mettre en lumière les limites rencontrées et les perspectives d'évolution du système.

1 Présentation générale et cadrage du projet

1.1 Analyse des besoins et objectifs stratégiques

véhicules autonomes reposent sur leur capacité à percevoir et à comprendre leur environnement de manière continue et instantanée. Pour garantir la sécurité critique des usagers, le système embarqué doit fonctionner avec une latence faible, car la moindre non-détection représente un risque potentiel d'accident. De ce fait, le taux de faux négatifs doit tendre vers zéro. Cette exigence de sécurité oblige le véhicule à localiser et identifier les piétons ainsi que les autres véhicules en temps réel.

Pour entraîner les modèles d'intelligence artificielle nécessaires à cette perception, l'approche centralisée classique sur le Cloud fait aujourd'hui face à deux verrous majeurs. Le premier concerne la bande passante : un seul véhicule autonome génère plusieurs téraoctets de données par jour, rendant l'envoi massif de ces flux bruts vers un serveur Cloud techniquement et économiquement irréalisable à grande échelle. Le second défi touche à la confidentialité et à la sécurité des données. Les images capturées en conditions réelles contiennent des informations sensibles telles que des visages, des plaques d'immatriculation ou des coordonnées de localisation précises. Leur centralisation est donc fortement restreinte par des réglementations strictes ainsi que par les politiques de confidentialité industrielles.

Le problème central de ce projet peut être formulé ainsi : **comment entraîner un modèle de détection performant sur des données distribuées entre véhicules sans jamais centraliser les images brutes sur un serveur, et sans dégrader les performances globales du système ?**

La solution explorée et apportée tout au long de ce travail est dans la mise en œuvre du *Federated Learning* couplé à l'algorithme d'agrégation.

1.2 Cahier des charges et exigences du sujet

Le sujet de projet nous a été fourni avec un ensemble d'objectifs, de composants système et de livrables attendus. Cette section en présente une synthèse structurée, avant d'expliquer

dans la section suivante comment chacune de ces exigences a été traduite en choix techniques concrets.

1.2.1 Priorisation des exigences : La Matrice Moscow

Pour fixer les priorités dès le début du projet, nous avons classé les exigences du cahier des charges avec la méthode MoSCoW (*Must have, Should have, Could have, Won't have*).

Vision Model: Deploy a YOLO algorithm on the boards to detect bicycles, cars, and pedestrians. Distributed Network: Create a federated learning architecture between the PC server and the edge boards (Jetson and Raspberry Pi). Data Privacy: Keep the KITTI images locally on the boards. No images must travel over the network. Data Fusion: Use the <code>FedAvg</code> algorithm to average the weights sent by the boards. MUST	AI Monitoring: Track mAP and Loss scores at each round to verify that the AI is learning correctly. Hardware Monitoring: Measure CPU usage, RAM usage, computation time, and the power consumption of the boards. Custom Training: Be able to configure a different number of local epochs from one board to another based on hardware performance. SHOULD
COULD	WON'T

Configuration File: Create a JSON file on the server to change hyperparameters without modifying the code. Fair Data Distribution: Split the images equally among the boards (quasi-IID approach). Alternative Algorithm: Test the FedProx algorithm to see if it handles the differences between the boards better.	Network Security: Encrypt the gRPC channel with SSL/TLS certificates (not enough time). Graphical Interface: Create a real-time visual application for the end user.
--	--

Figure 1 : Matrice MoSCoW

1.2.2 Livrables contractuels attendus

Pour valider la réussite de ce projet, le système final doit s'accompagner d'une série de livrables précis :

- **Un système FL fonctionnel :** Une infrastructure réseau opérationnelle en environnement réel et simulé.
- **Un modèle de détection entraîné :** Un réseau de neurones optimisé pour la reconnaissance des véhicules et des piétons.

- **Une démonstration d'inférence :** La preuve visuelle du bon fonctionnement de la détection directement sur le matériel Edge embarqué.
- **Un rapport technique complet :** Ce document couvrant l'architecture système, les paramètres FL, les résultats d'évaluation, la comparaison entre l'entraînement centralisé et fédéré, ainsi que l'analyse des limites et perspectives.

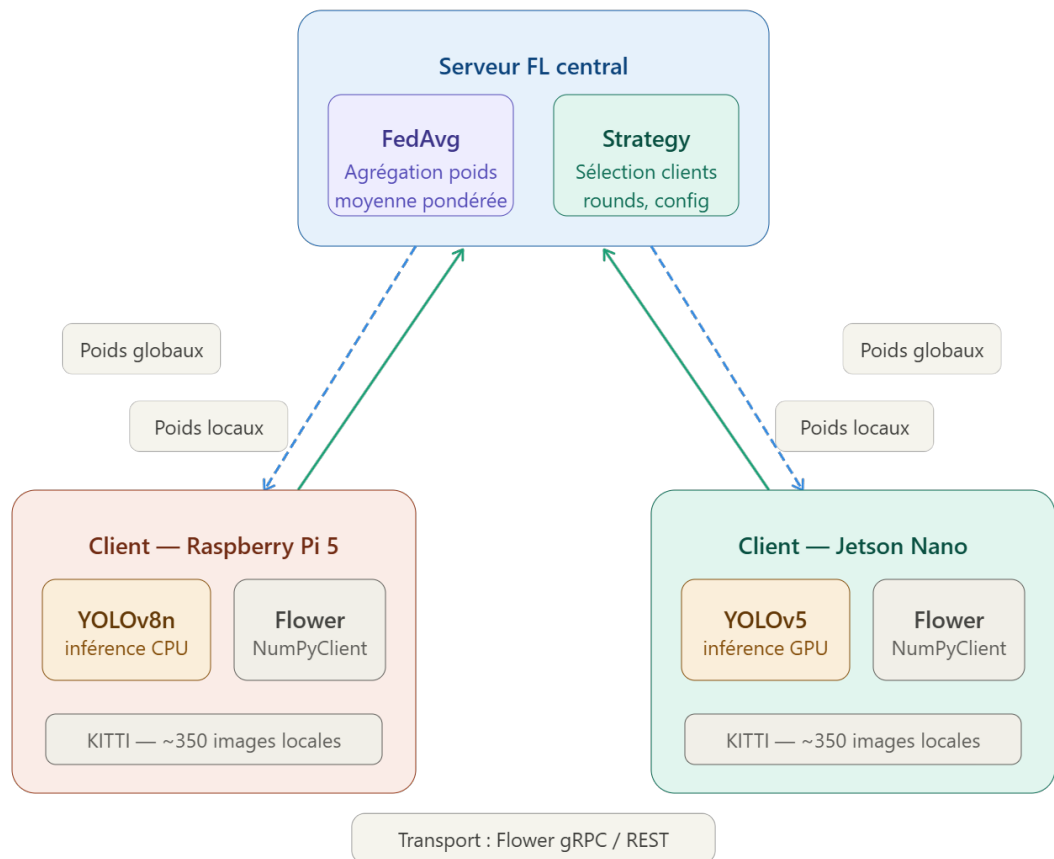
L'objectif principal de ce projet est de concevoir, d'implémenter et d'évaluer un système complet d'Apprentissage Fédéré (*Federated Learning* - FL) de bout en bout. Ce système est dédié à la détection en temps réel de véhicules et de piétons, avant d'être déployé sur des cibles matérielles contraintes (*Edge devices*).

2 Conception de l'architecture distribuée

2.1 Topologie du réseau d'Apprentissage Fédéré (FL)

Le système développé repose sur une architecture de *Federated Learning* (FL) à deux niveaux : un serveur central chargé de l'agrégation des modèles et trois clients embarqués. Cette organisation permet d'entraîner des modèles de détection d'objets sans centraliser les données brutes.

- **Serveur FL** : Il est exécuté sur un ordinateur local et orchestre l'ensemble du processus en s'appuyant sur le framework Flower. À chaque round, il gère la connexion et la participation des cartes, distribue les poids du modèle global aux clients connectés, attend la réception de leurs poids locaux mis à jour, puis applique l'algorithme d'agrégation pour générer le modèle global.
- **Cartes embarqués (Clients FL)** : Composés de modules Raspberry Pi et Jetson Nano, ces nœuds matériels sont connectés en réseau local avec le serveur. Chaque carte a la charge d'exécuter l'entraînement local du modèle sur ses propres données locales.



2.2 Apprentissage centralisé : Établissement du modèle de référence

Avant de concevoir l'infrastructure distribuée, nous avons débuté le projet par une phase de compréhension et de prise en main de l'écosystème YOLO à travers une approche d'apprentissage centralisé.

Cette étape préliminaire consistait à regrouper l'intégralité du sous-ensemble de données KITTI (soit les 700 images annotées) sur une seule et unique machine physique performante pour y entraîner localement le modèle de vision.

L'objectif de cette manipulation était double :

Valider la pipeline de détection : S'assurer que le format des annotations, les hyperparamètres de l'architecture YOLO étaient parfaitement fonctionnels et exempts d'erreurs.

Définir un point de comparaison : Obtenir les scores maximaux théoriques (mAP et vitesse de convergence) du modèle lorsqu'il a un accès direct et illimité à toutes les données. Ces mé-

triques "centralisées" nous ont servi de référentiel absolu pour évaluer, par la suite, la perte ou le maintien d'efficacité induit par le cloisonnement des données de l'apprentissage fédéré.

Pour cette phase d'apprentissage centralisé initial, notre choix s'est porté sur le modèle **YOLOv8n** (version *nano*). Ce choix se justifie par l'excellent compromis théorique qu'offre cette architecture de dernière génération entre une taille de modèle extrêmement réduite — idéale pour les contraintes de stockage des futurs systèmes embarqués — et une vitesse d'inférence élevée, indispensable pour garantir une détection de véhicules et de piétons en temps réel sans compromettre la précision globale.

2.3 Phase de prototypage : de la simulation au client unique

Le développement du système a suivi une progression rigoureuse par phases afin de valider chaque couche de complexité.

Phase 1 : Simulation sur ordinateur

Cette première étape consistait à simuler l'intégralité du système FL sur un unique ordinateur, sans matériel embarqué, pour s'assurer que la logique d'entraînement fédéré était correcte. Concrètement, deux clients Flower ont été instanciés sous forme de processus distincts sur la même machine. Chaque client disposait d'une moitié du dataset (700 images chacun). Cette phase a permis de valider :

- La logique d'entraînement locale et la transmission des poids au serveur.
- Le bon fonctionnement de l'algorithme FedAvg (moyenne pondérée).
- La cohérence mathématique de la fusion des poids sans contraintes réseau.

Phase 2 : Déploiement sur Raspberry Pi 5 (client unique)

La deuxième étape a marqué le passage sur du matériel réel en connectant une première carte embarquée (la Raspberry Pi 5) au serveur Flower exécuté sur le PC. Pour cette phase, le modèle YOLOv8 a été utilisé afin de tester l'inférence et l'apprentissage sur une cible physique.

L'utilisation d'un seul client sur le réseau local a permis de simplifier la recherche d'erreurs et de stabiliser les connexions. Cette phase nous a apporté plusieurs enseignements et validations :

- **Prise en main de Flower et YOLO** : Elle a permis de découvrir et de valider les communications simples avec Flower, ainsi que de tester concrètement le comportement d'un modèle YOLO directement sur l'architecture de la Raspberry Pi.
- **Contrôle à distance (SSH)** : C'était l'occasion de découvrir et de configurer la prise en main à distance de la carte via des liaisons SSH à partir d'un PC hôte, facilitant l'exécution des scripts sans écran dédié.
- **Performances matérielles** : Nous avons pu remarquer que la vitesse d'exécution de la Raspberry Pi 5 est remarquable pour de l'embarqué, notamment grâce à la puissance de son nouveau processeur.
- **Gestion de la mémoire (Workers)** : En cherchant à optimiser le chargement des données, nous avons testé le nombre maximal de processus en parallèle (*workers*). Cela a provoqué nos premiers plantages pour manque de mémoire (*Out of Memory*), ce qui nous a permis de comprendre les limites de configuration de la RAM de la carte.
- **Résolution de la surchauffe** : Lors des phases de calcul intensif, nous avons rencontré nos premiers problèmes de surchauffe sur la carte. Ce point a été corrigé en installant la Raspberry Pi dans un boîtier officiel ventilé.
- **Ajustement de l'alimentation électrique** : Nous avons constaté un manque de puissance électrique lors de l'utilisation d'un câble USB Type-C classique (la carte bridait ses performances ou affichait des alertes). Nous avons résolu ce problème en utilisant l'alimentation officielle Raspberry Pi capable de fournir 5A au maximum.

3 Gestion des données et cycle d'apprentissage

3.1 Évaluation initiale et limites de YOLOv8n

Initialement, notre choix s'est porté sur le modèle YOLOv8n pour ses performances théoriques optimales, offrant une taille réduite ainsi qu'un excellent compromis entre vitesse et précision. Comme détaillé précédemment dans la Phase 2 (Section 2.3), la Raspberry Pi 5 a servi de "sandbox" technologique.

Ce jalon a été stratégique car il a permis de découpler les problèmes : avant de confronter le système à l'hétérogénéité du réseau, nous avons utilisé YOLOv8n pour valider la robustesse du protocole de communication Flower. Cette étape a prouvé que l'architecture gRPC était capable de supporter le transfert de poids d'un modèle moderne, tout en nous permettant de définir nos standards de gestion thermique et électrique en conditions réelles d'inférence

3.2 Migration technologique vers YOLOv5

Cependant, lors de la Phase 3 visant à étendre le réseau à l'ensemble des cibles physiques (intégration des deux cartes NVIDIA Jetson Nano), un verrou technique critique est apparu.

L'environnement système officiel des Jetson Nano (**JetPack 4.6.1**) impose une version de Python figée à **3.6.9**. Or, la bibliothèque officielle de YOLOv8 (ultralytics) requiert impérativement $\text{Python} \geq 3.8$. De plus, le support de l'accélération graphique (CUDA 10.2) sur la Jetson Nano

est strictement lié à des versions spécifiques et figées de **PyTorch (1.10)** et **Torchvision**, incompatibles avec les exigences logicielles des versions récentes de YOLOv8. Toutes nos tentatives pour forcer l'installation de YOLOv8 ou pour mettre à jour l'environnement Python ont brisé la compatibilité avec les pilotes CUDA, rendant l'utilisation du GPU impossible.

Face à cette contrainte matérielle indépassable, nous avons opéré un **pivot technologique vers YOLOv5**. Ce modèle est pleinement compatible avec Python 3.6.9 ainsi qu'avec le couple PyTorch 1.10 / CUDA 10.2 fourni par NVIDIA pour la Jetson Nano. YOLOv5 offre des performances tout à fait adaptées pour la détection de véhicules et de piétons. Pour conserver l'homogénéité du réseau d'apprentissage, la Raspberry Pi 5 a également été basculée sur YOLOv5. Ce choix a garanti une intégration fluide, un accès complet à l'accélération matérielle GPU, et une exécution fonctionnelle sur l'ensemble de nos cibles hétérogènes.

4 Gestion des données et cycle d'apprentissage

4.1 Évaluation et Analyse des Résultats

La validation et l'évaluation des performances du système s'appuient sur le jeu de données de référence KITTI. Afin de simuler des conditions d'apprentissage stables, le volume de données a été segmenté de manière quasi-IID (*Independent and Identically Distributed*).

Le jeu de données est ainsi partitionné équitablement entre les trois nœuds physiques actifs du réseau (la Raspberry Pi et les deux Jetson Nano), ce qui représente 200 images d'entraînement par appareil. Ce choix respecte la philosophie fondamentale du *Federated Learning*, où les données brutes restent stockées localement sur chaque *Edge device*.

Pour gérer efficacement la distribution de ces lots d'images et automatiser leur transfert vers les différentes cartes embarquées, nous avons conçu des pipelines de commandes associant les outils « find », « head » et « rsync ».

4.2 Implémentation du protocole Flower et de l'algorithme FedAvg

4.2.1 Configuration de l'infrastructure réseau avec Flower

L'implémentation logicielle s'appuie sur Flower pour la gestion du réseau gRPC et des cycles. Côté serveur, la stratégie est instanciée avec les paramètres de contrôle suivants :

$\text{fraction_fit} = 1.0$: Tous les clients disponibles participent à chaque round d'entraînement.

$\text{Fraction_evaluate} = 0$: Évaluation systématique à l'issue de chaque round.

$\text{min_available_client} = 3$: Le serveur bloque et attend que les deux Jetson Nano (« comvis » et « js1 ») et la Raspberry Pi soient connectées avant de lancer le premier round.

4.2.2 Logique algorithmique de la fusion globale (FedAvg)

Le cœur algorithmique de la fusion repose sur un script Python personnalisé (*fusion.py*) qui implémente la logique d'agrégation **FedAvg** (*Federated Averaging*). À chaque fin de round, les clients transmettent leurs poids locaux mis à jour sous forme de tableaux NumPy ainsi que leurs métriques associées (loss, nombre d'exemples). Le serveur réalise alors une moyenne pondérée, couche par couche, de ces poids synaptiques proportionnellement au volume de données de chaque client, avant de redistribuer le modèle global mis à jour.

Le tableau suivant résume la configuration finale de notre banc d'essai :

Composant	Rôle	Modèle	Framework	Dataset local
Serveur (PC)	Agrégation FL	—	Flower + FedAvg	—
Jetson Nano comvis	Client FL 1	YOLOv5	Flower + PyTorch	200 images KITTI
Jetson Nano js1	Client FL 2	YOLOv5	Flower + PyTorch	200 images KITTI
pi_01	Client FL 3	YOLOv5	Flower + PyTorch	200 images KITTI

Tableau 1 – Configuration du système Federated Learning déployé sur matériel réel

4.2.3 Pilotage dynamique via fichier de configuration serveur

Pour éviter de figer les hyperparamètres dans le code source et permettre une gestion agile du réseau, nous avons mis en place un fichier de configuration centralisé au format JSON (« `confid_server.json` ») côté serveur. Ce fichier permet à l'administrateur du réseau de définir dynamiquement, et pour chaque client de manière individualisée, les conditions de l'entraînement fédéré :

Le nombre de rounds globaux : Définit le cycle total d'itérations de l'apprentissage fédéré.

Le nombre minimal de clients requis : Spécifie le seuil de cartes connectées nécessaire pour que le serveur lance officiellement le processus d'apprentissage (évitant ainsi de démarrer si un périphérique est hors ligne).

Le nombre d'époques locales (`epochs`) : Spécifie le nombre de passes d'entraînement que chaque carte doit effectuer sur son propre GPU ou CPU avant d'envoyer ses poids au serveur.

La taille des lots (`batch_size`) : Ajuste le volume d'images traitées simultanément lors d'une passe, un paramètre critique pour le contrôle et l'évitement de la saturation de la mémoire sur l'embarqué.

Le nombre de sous-processus (`workers`) : Contrôle l'allocation des ressources CPU pour le chargement en parallèle des images.

Ce fichier intègre également une entrée `"default"`. Elle garantit l'évolutivité du système : si un nouveau client inconnu se connecte au réseau gRPC, le serveur lui attribue automatiquement ces métriques par défaut, sans nécessiter l'arrêt du serveur ni la modification des scripts.

4.2.4 Protocole d'exécution et arguments de la commande client

Le lancement de l'entraînement sur les périphériques Edge s'effectue via un script Python (« `client_jetson.py` » ou « `client_pi.py` ») exécuté en ligne de commande. Afin de rendre l'implémentation générique et adaptable à chaque nœud, nous avons développé un système d'arguments modulaires.

Une commande type se présente ainsi :

```
python3 client_jetson.py --data /home/comvis/kitti/data.yaml --server 192.168.0.10X:8080 --  
device-type jetson --device-id comvis
```

Chaque attribut joue un rôle précis dans l'initialisation du nœud :

--data : Spécifie le chemin absolu vers le fichier de configuration du dataset (data.yaml). Cela permet à la carte de localiser ses images locales et de connaître le nombre de classes à détecter (ici, réduction de 80 à 8 classes pour l'adaptation à KITTI).

--server : Indique l'adresse IP et le port d'écoute du serveur Flower central (IP : PORT). C'est cette cible que le canal gRPC va tenter de joindre pour s'enregistrer dans le round.

--device-type : Précise la nature du matériel (jetson ou raspberrypi). Cet argument est crucial car il permet au script de configurer automatiquement l'environnement matériel, notamment en activant l'accélération graphique cuda :0 sur les Jetson ou en forçant l'exécution sur cpu pour la Raspberry Pi.

--device-id : Attribue un identifiant unique au client (comvis, js1, etc.). Cet identifiant permet au serveur de reconnaître formellement la carte qui communique avec lui et de lui appliquer sa configuration personnalisée issue du fichier JSON.

5 Résolution des problèmes et optimisations matérielles

5.1 Scénario de mise en route du réseau physique

Une fois l'environnement YOLOv5 et Flower stabilisé, nous avons procédé au déploiement réel. Le protocole consistait à lancer le script serveur sur le PC, ouvrant un port de communication gRPC, puis à exécuter simultanément les scripts clients sur les cartes embarquées.

5.2 Analyse des anomalies et crashes matériels

Dès le lancement des premiers rounds d'entraînement sur le réseau de cartes, nous avons été confrontés à deux pannes critiques qui bloquaient le système :

5.2.1 Saturation de la mémoire RAM (Erreur "Cannot allocate memory")

Sur les cartes Jetson Nano, le script client parvenait à s'enregistrer auprès du serveur gRPC mais crashait instantanément au moment d'initier la première époque de l'entraînement local, renvoyant l'erreur systématique : `OSError: [Errno 12] Cannot allocate memory`. L'analyse approfondie des ressources système a révélé deux causes majeures :

- **La fragmentation de la mémoire :** Au repos, l'environnement graphique et les services de base de JetPack consomment déjà une part importante des 4 Go de RAM de la Jetson. Après plusieurs tests ou interruptions brutales de scripts, la mémoire se fragmente.
- **Le mécanisme ZRAM insuffisant :** Par défaut, l'architecture JetPack configure un espace de SWAP virtuel compressé en RAM (ZRAM) limité à 1,9 Go. Lors du chargement du modèle YOLOv5 en mémoire et de l'allocation des sous-processus de chargement de données (*workers*), cette barrière est immédiatement franchie, provoquant un crash par débordement de mémoire (OOM - *Out Of Memory*).

5.2.2 Instabilité électrique et déconnexions matérielles

Sur la Jetson « comvis », la carte se déconnectait purement et simplement du réseau local (perte instantanée des liaisons SSH et de la session Flower) quelques secondes après le top départ de l'entraînement. Ce phénomène est lié à une chute de tension électrique critique. Au repos, la Jetson présente une consommation modérée de 2 à 3W. Dès que le GPU CUDA et le CPU sont sollicités à 100% pour calculer les gradients de YOLOv5, la demande énergétique grimpe instantanément au-delà des 10W. L'alimentation via le port Micro-USB classique ne parvenant pas à encaisser ce pic d'appel de courant, la tension s'effondre et la carte se met en sécurité matérielle en coupant ses interfaces ou en redémarrant.

5.2.3 Pertes de liaison récurrentes liées à l'infrastructure réseau (Routeur)

En dehors des pannes purement énergétiques, le banc d'essai a souffert de déconnexions gRPC très fréquentes au niveau de la couche réseau. Le routeur utilisé pour l'interconnexion locale gérait difficilement le maintien des connexions asynchrones persistantes requises par le framework Flower. Lors des phases de calcul local où le canal gRPC restait "silencieux" (le temps que les cartes finissent leurs époques), le routeur fermait prématurément les sockets ou subissait des micro-coupures. Ces instabilités provoquaient l'exclusion immédiate des clients par le serveur, forçant l'interruption complète du round fédéré en cours.

5.2.4 Hétérogénéité logicielle native entre Raspberry Pi et Jetson Nano

L'intégration simultanée de cibles hétérogènes a mis en lumière un conflit majeur de versions logicielles natives :

La Raspberry Pi 5 dispose d'un système d'exploitation récent fournissant nativement une version moderne de Python (≥ 3.10), ce qui permettait d'exécuter initialement YOLOv8 de manière fluide.

À l'inverse, la Jetson Nano est bridée par son architecture constructeur JetPack 4.6.1, qui impose une version obsolète et figée de Python 3.6.9.

Cette divergence profonde a rendu impossible l'utilisation d'un code client unique et standardisé. La bibliothèque officielle de YOLOv8 (ultralytics) refusant catégoriquement de s'installer sur Python 3.6, et l'accélération GPU CUDA (10.2) de la Jetson étant intimement liée à des versions figées de PyTorch (1.10) et Torchvision, nous avons été confrontés à un véritable mur de compatibilité.

5.2.5 Complexité de réplique de l'environnement sur une nouvelle Jetson

Lors de l'introduction d'une seconde Jetson Nano pour étendre notre réseau à trois clients, la mise en place de la solution s'est avérée particulièrement laborieuse. Reconstituer exactement le même écosystème logiciel (alignement strict des versions de chaque bibliothèque, de Flower, de PyTorch, de Torchvision et des dépendances de traitement d'images) représente un défi complexe sur ces architectures contraintes.

Pour contourner l'enfer des dépendances globales et garantir un alignement miroir parfait avec la première Jetson, nous avons dû isoler l'application en passant par un environnement virtuel (venv). L'installation a nécessité de compiler manuellement certaines roues (*wheels*) Python spécifiques à l'architecture ARM64 de NVIDIA, afin de s'assurer que le comportement algorithmique et la structure des poids exportés sous forme de tableaux NumPy restent rigoureusement identiques.

6 Déploiement, Évaluation et Analyse des Résultats

6.1 Solutions techniques et configurations appliquées sur les cartes embarqués

Pour résoudre les verrous du Chapitre 5 et stabiliser notre banc d'essai, nous avons appliqué trois optimisations majeures :

- **Création d'un SWAP physique persistant** : Nous avons alloué un fichier d'échange (SWAP) de 4 Go directement sur la carte SD de la Jetson Nano pour épauler la RAM. Pour éviter que cette configuration ne s'efface à chaque coupure ou redémarrage, nous l'avons inscrite de manière permanente dans le descripteur de fichiers système `/etc/fstab` via la commande : `echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab`. Au repos, la commande `free -h` valide une mémoire totale disponible étendue (RAM + SWAP) et une RAM utilisée stabilisée à seulement 1.0 Go.

6.2 Évaluation des performances après optimisation

Une fois le matériel stabilisé, nous avons mené un entraînement complet. Les performances ont été enregistrées dans un fichier CSV à chaque round.

6.2.1 Pipeline de collecte des données et génération des rapports CSV

L'analyse des performances présentée dans ce chapitre ne repose pas sur des estimations théoriques, mais sur l'exploitation de données réelles récoltées directement sur le terrain à chaque cycle d'apprentissage.

Pour ce faire, nous avons instrumenté nos scripts clients afin de mesurer et d'extraire les métriques physiques et algorithmiques à la fin de chaque round Flower. Côté client, un module de supervision enregistre en continu :

- **Les métriques de performance de l'IA** : La valeur de la *loss* locale et le score mAP calculés sur le jeu de validation de la carte.

- **Les métriques matérielles :** Le temps d'entraînement local exact (en secondes), le taux d'utilisation moyen du CPU, la quantité de mémoire RAM consommée, ainsi que le volume de données réseau transité (octets émis et reçus via gRPC).

À la fin de chaque round, chaque client structure ces données et les exporte de manière autonome. Ces informations sont téléversées et centralisées sur le serveur dans un fichier d'historique au format CSV. Ce fichier de métriques consolidé constitue notre base de données analytique. C'est à partir de ce document que nous avons pu tracer les courbes de convergence et dresser les tableaux comparatifs d'efficacité CPU/GPU présentés ci-après.

6.2.2 Évolution des métriques de l'IA (mAP et Loss)

L'analyse montre une progression cohérente du modèle global :

- La mAP@50 progresse de manière régulière au fil des rounds, passant en moyenne de 0,0245 (round 1) à 0,0708 (round 4), soit une amélioration de **+189 %**. Les valeurs de mAP@50-95 suivent la même tendance, passant de 0,0056 à 0,0250. Bien que ces valeurs absolues restent faibles (ce qui s'explique par un faible nombre de rounds testés, une seule époque locale par round epochs=1 et un dataset restreint à 700 images), la courbe valide mathématiquement la convergence de l'apprentissage fédéré.
- La loss décroît de 0,1974 (round 1) à 0,1419 (round 3), soit une réduction de **-28,1 %**. L'écart de loss entre les deux Jetson à chaque round est quasi nul (< 0,0001), ce qui valide la cohérence de l'algorithme FedAvg sur des configurations client identiques.

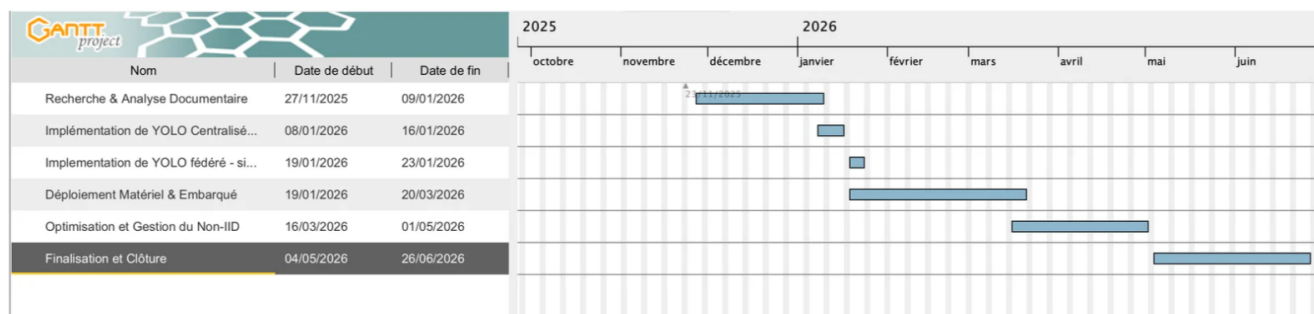
6.2.3 Comportement et performances matérielles

L'utilisation CPU moyenne des Jetson Nano est restée modérée (entre 13 % et 21 %). L'occupation RAM s'est stabilisée entre 1 350 et 2 382 Mo selon les phases. On observe cependant une hétérogénéité temporelle : la carte « comvis » présente un temps d'entraînement local systématiquement supérieur à la carte « js1 » (730–806 s contre 451–512 s). Cela est très probablement dû au refroidissement de la « comvis » qui est bien plus performant.

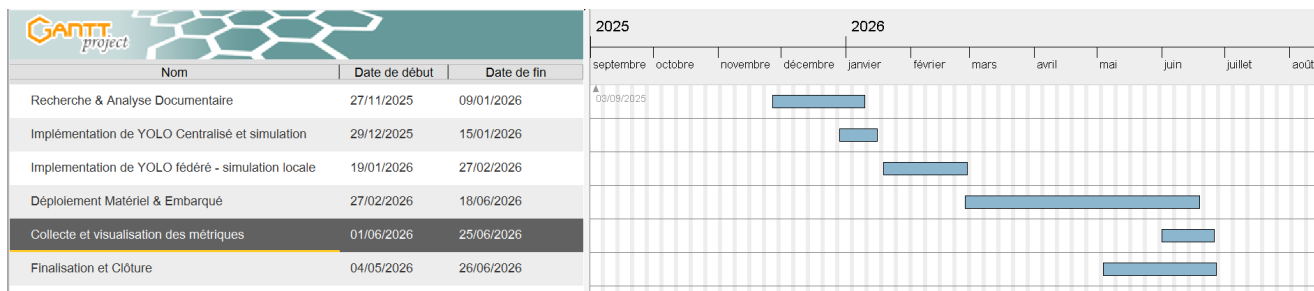
7 Bilan de gestion de projet

7.1 Suivi, planification temporelle et jalons (Gantt)

Concernant la gestion de projet du projet, nous avons mis en place, au début du projet, un Gantt pour gérer le planning (ci-dessous).



Nous pouvons comparer le Gantt réalisé au début du projet ainsi que celui réalisé réellement durant le projet, nous observons qu'ils sont différents notamment dû au problèmes rencontrés niveau version durant la mise en place de la phase 3.



7.2 Organisation et communication

Afin d'assurer la transparence et la bonne gestion du projet, nous avons maintenu une communication régulière avec notre tuteur à travers des réunions de suivi périodiques. Ces échanges ont été l'occasion de présenter l'avancement de notre travail et de valider les grandes étapes du projet.

De plus, la rédaction fréquente de rapports de progression nous a permis de formaliser le suivi des tâches (réalisées et planifiées), garantissant ainsi le plus possible respect de notre Gantt.

8 Conclusion

Ce projet, réalisé en collaboration avec l'université vietnamienne HUST , a permis de créer et de tester un système complet d'apprentissage fédéré (Federated Learning) pour détecter des véhicules et des piétons. Cette approche sur terminaux embarqués (Edge Computing) évite d'envoyer les images brutes sur un serveur Cloud, ce qui résout les problèmes de bande passante et de confidentialité. Les données du dataset KITTI restent ainsi stockées localement sur les cartes. Nous avons validé la logique d'entraînement et la fusion des poids avec l'algorithme FedAvg d'abord en simulation , puis sur un réseau physique composé d'un serveur central et de trois clients : une Raspberry Pi 5 et deux NVIDIA Jetson Nano. Le serveur orchestre l'ensemble du processus via le framework Flower et un script personnalisé appelé fusion.py.

Le passage sur le matériel réel a mis en avant plusieurs difficultés physiques propres aux systèmes embarqués. L'entraînement intensif a demandé des corrections matérielles pour gérer la surchauffe et les baisses de tension des cartes. Pour corriger les crashes de mémoire (RAM) sur les Jetson Nano, nous avons dû ajouter un fichier de SWAP. De plus, le système JetPack 4.6.1 des Jetson Nano bloque Python en version 3.6.9, ce qui a empêché d'utiliser YOLOv8. Nous avons donc choisi de passer tout le réseau sur YOLOv5. Ce choix a permis d'utiliser l'accélération graphique GPU avec CUDA tout en gardant un modèle identique entre la Raspberry Pi et les Jetson pour que la fusion des poids fonctionne.

Les résultats montrent que le modèle global progresse, comme le prouvent la baisse de la perte (*loss*) et la hausse de la précision mAP@50 au fil des rounds. Pour la suite du projet, plusieurs perspectives importantes se dégagent. Tout d'abord, il serait utile de créer un pack d'installation automatisé pour installer toutes les bibliothèques et configurations d'un seul coup sur une nouvelle carte. Ensuite, l'utilisation de cartes Jetson plus récentes permettrait de disposer d'une version de Python moderne et ainsi de passer à un modèle YOLO plus récent et performant. Enfin, sur le plan logiciel, il est nécessaire de développer un code pour gérer les pannes : si une carte subit un crash en plein entraînement, ce système lui permettrait de se reconnecter et de reprendre directement au round où elle s'est arrêtée, sans perdre la progression des autres clients et sans devoir tout redémarrer.